

Guia Reprodutível para Redução do Ansatz de Dicke

Redução estrutural sem Jacobiano e compressão geométrica J90 passo a passo

Marlon Maxuel Souza Mendes

16 de setembro de 2026

Sumário

1	Objetivo deste documento	3
2	Problema físico e notação	3
3	A construção nominal do circuito FULL	4
4	Método 1 — redução estrutural sem Jacobiano	4
4.1	Ideia central	4
4.2	Triangulação combinatória antes do Jacobiano	5
4.3	Exemplo completo: $n = 10$, $k = 4$	6
4.4	Passo a passo computacional da redução estrutural	7
4.5	Até onde a redução estrutural foi testada	9
4.6	Auditoria física de recursos	9
5	Resumo operacional do Método 1	10
6	Método 2 — compressão geométrica J90	10
6.1	Passo 1 — usar apenas o circuito REDUCED	10
6.2	Passo 2 — coordenada angular normalizada	11
6.3	Passo 3 — definição do Jacobiano	11
6.4	Passo 4 — estimativa finite-shot por diferenças centradas	11
6.5	Passo 5 — duas réplicas independentes	12
6.6	Passo 6 — repetir a compilação em várias seeds	12
6.7	Passo 7 — projeção PSD	12
6.8	Passo 8 — escolher r_{90}	13
6.9	Passo 9 — resultados de dimensão	13
6.10	Passo 10 — nova coordenada q	13
6.11	Passo 11 — construir J90-DIRECT	13
6.12	Passo 12 — nunca confundir ordem geométrica com ordem do circuito	14
6.13	Passo 13 — hard gate de equivalência	14
6.14	Passo 14 — entender o que foi e não foi reduzido	15
7	Benchmark necessário depois de cada redução	15
7.1	Otimizadores usados	15
7.2	Efeito de FULL $\rightarrow$ REDUCED	16
7.3	Efeito de REDUCED $\rightarrow$ J90	16
8	A triangulação final da redução	16
9	Checklist	17

10 Algoritmo resumido	18
11 resultado	20
12 Conclusão	20

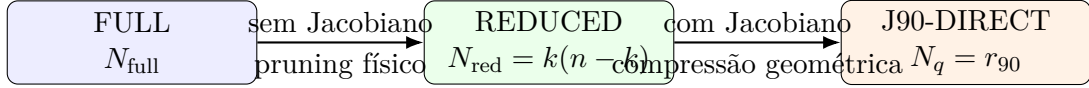
1 Objetivo deste documento

A ideia é reconstruir os resultados sem precisar conhecer previamente toda a sequência histórica do projeto.

A pergunta é:

Como reduzir o número de parâmetros de um ansatz de Dicke sem destruir a família física que queremos explorar e como verificar, experimentalmente, se a redução ajuda a otimização?

Existem **duas reduções diferentes**.



A primeira redução é

$$\boxed{\text{FULL} \longrightarrow \text{REDUCED},}$$

e pode ser feita **sem calcular o Jacobiano**.

A segunda é

$$\boxed{\text{REDUCED} \longrightarrow \text{J90-DIRECT},}$$

e depende de uma estimativa shot-based do Jacobiano.

A distinção é fundamental:

$$\boxed{\text{REDUCED remove motifs físicos;}}$$

$$\boxed{\text{J90 reduz graus de liberdade, mas preserva os motifs do REDUCED.}}$$

2 Problema físico e notação

Trabalhamos no setor de peso de Hamming fixo:

$$\mathbf{x} = (x_1, \dots, x_n) \in \{0, 1\}^n, \quad \sum_{i=1}^n x_i = k.$$

O conjunto factível é

$$\Omega_{n,k} = \left\{ \mathbf{x} \in \{0, 1\}^n : \sum_i x_i = k \right\},$$

e possui

$$\boxed{M = \binom{n}{k}}$$

configurações.

O estado inicial uniforme é o estado de Dicke:

$$\boxed{|D_n^k\rangle = \frac{1}{\sqrt{\binom{n}{k}}} \sum_{\mathbf{x} \in \Omega_{n,k}} |\mathbf{x}\rangle.}$$

Para uma função diagonal,

$$C(\theta) = \sum_{x \in \Omega_{n,k}} E_x p_x(\theta).$$

O circuito variacional modifica as probabilidades p_x permanecendo no setor factível.

3 A construção nominal do circuito FULL

A família de circuitos estudada começa em

$$|0\rangle^{\otimes(n-k)} |1\rangle^{\otimes k}.$$

Portanto os k últimos qubits estão inicialmente ocupados.

Definimos

$$\mathcal{O} = \{n-k, n-k+1, \dots, n-1\}.$$

Chamaremos $\mathcal{O}$ de **bloco inicialmente ocupado**.

A construção percorre motifs parametrizados associados a pares (l, i) .

No código usado no projeto, o esqueleto é:

```
for l in range(n)[::-1]:
    for i in range(l-1, l-1-k, -1):
        if i < 0:
            continue

        # construir motif parametrizado associado ao par (l, i)
```

A parametrização nominal contém

$$N_{\text{full}} = kn - \frac{k(k+1)}{2}.$$

Essa expressão pode ser reescrita como

$$N_{\text{full}} = k(n-k) + \binom{k}{2}.$$

Essa decomposição é o primeiro indício de que há dois setores diferentes.

4 Método 1 — redução estrutural sem Jacobiano

4.1 Ideia central

Nesta primeira redução não calculamos derivadas, SVD, Fisher ou Hessiana.

Usamos apenas:

1. a preparação inicial do estado;
2. os índices (l, i) dos motifs;
3. a posição do bloco inicialmente ocupado;
4. testes diretos de invariância do estado.

A regra candidata é extremamente simples.

Um motif é marcado como candidato a estruturalmente morto quando

$$l \in \mathcal{O} \quad \text{e} \quad i \in \mathcal{O}.$$

Ou seja:

$$\mathcal{D} = \{(l, i) : l, i \in \mathcal{O}\}.$$

Como existem k índices em $\mathcal{O}$, o número de pares distintos é

$$N_{\text{dead}} = \binom{k}{2} = \frac{k(k-1)}{2}.$$

Então a previsão para o número de motifs mantidos é

$$N_{\text{keep}} = N_{\text{full}} - N_{\text{dead}}.$$

Substituindo,

$$N_{\text{keep}} = k(n-k) + \binom{k}{2} - \binom{k}{2} \quad (1)$$

$$= \boxed{k(n-k)}. \quad (2)$$

Portanto a hipótese estrutural é

$$\boxed{N_{\text{red}} = k(n-k)}.$$

4.2 Triangulação combinatória antes do Jacobiano

Há uma relação importante que pode ser usada como **hipótese orientadora** antes de calcular qualquer Jacobiano.

O setor factível de peso k pode ser representado pelo grafo de Johnson $J(n, k)$.

Cada configuração factível tem exatamente

$$\boxed{\deg J(n, k) = k(n-k)}$$

vizinhos obtidos trocando um bit ocupado por um bit vazio.

Temos então três contagens:

$$\boxed{N_{\text{full}} = k(n-k) + \binom{k}{2}},$$

$$\boxed{\deg J(n, k) = k(n-k)},$$

e

$$\boxed{N_{\text{dead}} = \binom{k}{2}}.$$

Isso sugere a triangulação combinatória

$$\boxed{N_{\text{full}} - N_{\text{dead}} = N_{\text{red}} = \deg J(n, k) = k(n-k)}.$$

Hipótese/guia estrutural: A coincidência entre N_{red} e o grau $k(n-k)$ do grafo de Johnson não é, sozinha, uma prova matemática de que cada parâmetro ativo corresponde a uma aresta específica

do grafo. Ela funciona como uma hipótese estrutural forte: antes de calcular qualquer Jacobiano, prevê quantos parâmetros devem sobreviver.

Assim podemos formular uma regra **pré-Jacobiano**:

$$N_{\text{prune}}^{\text{hyp}} = N_{\text{full}} - \deg J(n, k) = \binom{k}{2}.$$

Esta é exatamente a quantidade que foi encontrada pela regra do bloco inicialmente ocupado.

4.3 Exemplo completo: $n = 10$, $k = 4$

Para

$$n = 10, \quad k = 4,$$

o bloco ocupado é

$$\mathcal{O} = \{6, 7, 8, 9\}.$$

A contagem nominal é

$$N_{\text{full}} = 4(10) - \frac{4(5)}{2} = 40 - 10 = 30.$$

Pela decomposição,

$$30 = 4(10 - 4) + \binom{4}{2} = 24 + 6.$$

Os seis pares inteiramente contidos em $\mathcal{O}$ são

$$(7, 6),$$

$$(8, 7), \quad (8, 6),$$

$$(9, 8), \quad (9, 7), \quad (9, 6).$$

Logo,

$$N_{\text{dead}} = 6,$$

e

$$N_{\text{red}} = 30 - 6 = 24.$$

Ao mesmo tempo,

$$\deg J(10, 4) = 4(10 - 4) = 24.$$

Portanto:

$$30_{\text{full}} \longrightarrow 24_{\text{reduced}}$$

é previsto sem calcular J .

4.4 Passo a passo computacional da redução estrutural

Passo 1 — construir o circuito FULL

Crie o circuito exatamente como definido pelo ansatz, sem remover nada.

Para cada motif, registre pelo menos:

- l ;
- i ;
- tipo de gate;
- período angular;
- ordem temporal do motif no circuito.

Exemplo:

```
records.append({
    "l": int(l),
    "i": int(i),
    "gate_type": gate_type,
    "period": float(period),
    "circuit_order": int(circuit_order),
})
```

Passo 2 — determinar o bloco inicialmente ocupado

```
occupied_start = n - k
occupied = set(range(occupied_start, n))
```

Para (10, 4):

```
occupied = {6, 7, 8, 9}
```

Passo 3 — classificar candidatos mortos

```
def is_dead_pair(n, k, l, i):
    occupied_start = n - k
    return (
        l >= occupied_start
        and i >= occupied_start
    )
```

Essa classificação é feita apenas pelos índices.

Não há Jacobiano.

Passo 4 — conferir a contagem

Exija

$$N_{\text{dead}} = \binom{k}{2}.$$

E

$$N_{\text{active}} = N_{\text{full}} - N_{\text{dead}} = k(n - k).$$

No código:

```

expected_dead = math.comb(k, 2)
expected_active = k * (n - k)

assert len(dead_idx) == expected_dead
assert len(active_idx) == expected_active

```

Passo 5 — teste de invariância sem derivadas

Este é o teste mais importante antes de realmente apagar os motivos.

Escolha um vetor global aleatório de parâmetros,

$$\boldsymbol{\theta}.$$

Construa um segundo vetor

$$\boldsymbol{\theta}',$$

mantendo todos os parâmetros ativos iguais e randomizando apenas os parâmetros candidatos a mortos.

Formalmente,

$$\theta'_j = \theta_j, \quad j \in \mathcal{A},$$

enquanto

$$\theta'_j$$

é escolhido independentemente para

$$j \in \mathcal{D}.$$

Calcule

$$|\psi(\boldsymbol{\theta})\rangle$$

e

$$|\psi(\boldsymbol{\theta}')\rangle.$$

Compare:

$$D_{\text{TV}} = \frac{1}{2} \sum_x |p_x(\boldsymbol{\theta}) - p_x(\boldsymbol{\theta}')|.$$

Também é útil medir a distância projetiva entre estados.

O teste deve ser repetido para muitas configurações globais aleatórias.

Passo 6 — controle positivo

Não basta observar que o setor candidato morto não muda o estado.

É necessário verificar se o teste consegue detectar uma mudança quando ela realmente existe.

Escolha um parâmetro classificado como ativo e perturbe apenas ele:

$$\theta_j \longrightarrow \theta_j + \delta.$$

Então exija que a distribuição ou o estado mude de forma não trivial.

Esse é o controle positivo.

Passo 7 — construir fisicamente o REDUCED

Somente depois dos testes anteriores, altere o construtor:

```
for l in range(n)[::-1]:
    for i in range(l-1, l-1-k, -1):
        if i < 0:
            continue

        if is_dead_pair(n, k, l, i):
            continue

        # construir apenas o motif ativo
```

O ponto importante é:

o gate morto não é mascarado;

ele simplesmente não é criado.

4.5 Até onde a redução estrutural foi testada

A regra foi levada aos seguintes regimes:

(n, k)	$M = \binom{n}{k}$	N_{full}	$\binom{k}{2}$	N_{red}	$\deg J(n, k)$
(8, 3)	56	18	3	15	15
(10, 4)	210	30	6	24	24
(12, 4)	495	38	6	32	32
(14, 5)	2002	55	10	45	45
(16, 6)	8008	75	15	60	60

Portanto, até o maior regime dessa campanha,

$$(n, k) = (16, 6),$$

foi possível prever a redução

$$75 \longrightarrow 60$$

sem calcular o Jacobiano.

A hipótese pré-Jacobiano produziu, nos cinco regimes testados, exatamente a contagem usada depois no circuito REDUCED.

4.6 Auditoria física de recursos

Depois de construir FULL e REDUCED, ambos devem ser transpiled para a mesma base e com o mesmo protocolo.

Na campanha atual foi utilizada a base

$$\{\text{rz}, \text{sx}, x, \text{cx}\}.$$

Com nível de otimização 3:

(n, k)	N_{full}	N_{red}	CX FULL	CX RED.	depth FULL/RED.
(8, 3)	18	15	182	160	377/333
(10, 4)	30	24	330	276	597/511
(12, 4)	38	32	422	368	757/671
(14, 5)	55	45	640	540	917/804
(16, 6)	75	60	900	740	1077/937

Logo a redução estrutural realmente diminui o circuito físico.

5 Resumo operacional do Método 1

Para reproduzir $\text{FULL} \rightarrow \text{REDUCED}$, o aluno deve executar:

- R1.1** escolher (n, k) ;
- R1.2** calcular $\mathcal{O} = \{n - k, \dots, n - 1\}$;
- R1.3** construir o FULL;
- R1.4** registrar todos os pares (l, i) ;
- R1.5** marcar como candidato morto todo par com $l, i \in \mathcal{O}$;
- R1.6** verificar que a contagem é $\binom{k}{2}$;
- R1.7** verificar que sobram $k(n - k)$ motifs;
- R1.8** comparar essa contagem com $\deg J(n, k) = k(n - k)$;
- R1.9** randomizar somente os candidatos mortos e verificar invariância;
- R1.10** perturbar um parâmetro ativo como controle positivo;
- R1.11** construir REDUCED pulando fisicamente os candidatos mortos;
- R1.12** transpilar FULL e REDUCED com a mesma base;
- R1.13** medir parâmetros, CX, tamanho e profundidade;
- R1.14** somente então executar o benchmark de otimização.

Até esse ponto **não é necessário calcular o Jacobiano**.

6 Método 2 — compressão geométrica J90

A segunda redução começa **depois** que o REDUCED já foi definido.

Agora temos

$$N_{\text{red}} = k(n - k)$$

parâmetros estruturalmente ativos.

A pergunta muda:

Mesmo entre essas direções físicas, será que todas produzem movimentos probabilísticos de intensidade comparável perto do estado de Dicke?

Essa pergunta exige o Jacobiano.

6.1 Passo 1 — usar apenas o circuito REDUCED

Nunca calcule o J90 no espaço FULL incluindo parâmetros estruturalmente mortos.

Primeiro reduza:

$$\text{FULL} \rightarrow \text{REDUCED}.$$

Depois construa

$$J_A \in \mathbb{R}^{M \times k(n-k)}.$$

6.2 Passo 2 — coordenada angular normalizada

Os motifs CY e CCY podem possuir períodos angulares diferentes.

Por isso usamos uma fase normalizada:

$$\phi_j = \frac{2\pi}{T_j} \theta_j,$$

onde T_j é o período do parâmetro físico daquele motif.

O ponto de Dicke fornece uma referência

$$\phi_D.$$

Definimos

$$\Delta\phi = \phi - \phi_D$$

localmente.

No ponto de referência,

$$\Delta\phi = 0.$$

6.3 Passo 3 — definição do Jacobiano

Para cada estado factível x e coordenada ativa j ,

$$J_{xj} = \frac{\partial p_x}{\partial \phi_j}.$$

Se

$$\mathbf{p} = \begin{bmatrix} p_1 & \cdots & p_M \end{bmatrix}^T,$$

então

$$J = \begin{bmatrix} \partial p_1 / \partial \phi_1 & \cdots & \partial p_1 / \partial \phi_d \\ \vdots & & \vdots \\ \partial p_M / \partial \phi_1 & \cdots & \partial p_M / \partial \phi_d \end{bmatrix},$$

com

$$d = k(n - k).$$

6.4 Passo 4 — estimativa finite-shot por diferenças centradas

Não usamos o Jacobiano exato para definir o circuito final do benchmark.

Para cada coordenada j , use

$$\hat{J}_{:j} = \frac{\hat{\mathbf{p}}(+h\mathbf{e}_j) - \hat{\mathbf{p}}(-h\mathbf{e}_j)}{2h}.$$

Na campanha atual foi usado

$$h = 0.08$$

na coordenada de fase normalizada.

Cada $\hat{\mathbf{p}}$ vem de counts finite-shot.

No protocolo principal,

$$N_{\text{shots}} = 10\,000$$

por circuit-batch.

6.5 Passo 5 — duas réplicas independentes

Se usarmos diretamente

$$\hat{J}^T \hat{J},$$

o ruído da própria estimativa contribui positivamente para a diagonal e pode inflar artificialmente a dimensionalidade.

Por isso geramos duas réplicas independentes:

$$\hat{J}_A, \quad \hat{J}_B.$$

Então usamos o cross-Gram simetrizado:

$$G_{\text{cross}} = \frac{1}{2} \left[\hat{J}_A^T \hat{J}_B + \hat{J}_B^T \hat{J}_A \right].$$

O objetivo é reduzir o viés de auto-ruído.

6.6 Passo 6 — repetir a compilação em várias seeds

Uma única realização de shots não é suficiente para definir uma sub-base robusta.

No perfil usado para o resultado principal foram usadas

$$S = 5$$

compilações independentes.

Para cada seed s obtemos

$$G_{\text{cross}}^{(s)}.$$

Depois:

$$\overline{G}_{\text{cross}} = \frac{1}{S} \sum_{s=1}^S G_{\text{cross}}^{(s)}.$$

6.7 Passo 7 — projeção PSD

Como a matriz é estimada com ruído, pequenos autovalores negativos podem aparecer numericamente.

Simetrizamos:

$$G \leftarrow \frac{1}{2}(G + G^T).$$

Diagonalizamos:

$$G = V \Lambda V^T.$$

E substituímos

$$\lambda_i < 0$$

por

$$0.$$

Isso produz uma aproximação PSD:

$$\boxed{G_{+.}}$$

6.8 Passo 8 — escolher r_{90}

Ordene os autovalores:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq 0.$$

Defina

$$\boxed{r_{90} = \min \left\{ r : \frac{\sum_{i=1}^r \lambda_i}{\sum_i \lambda_i} \geq 0.90 \right\} .}$$

Os primeiros r_{90} autovetores formam

$$\boxed{Q_{90} = \begin{bmatrix} \mathbf{v}_1 & \cdots & \mathbf{v}_{r_{90}} \end{bmatrix} .}$$

6.9 Passo 9 — resultados de dimensão

Na campanha multi-seed:

(n, k)	N_{red}	r_{90}	retenção	redução de graus de liberdade
(8, 3)	15	11	0.9032	15 $\rightarrow$ 11
(10, 4)	24	16	0.9037	24 $\rightarrow$ 16
(12, 4)	32	20	0.9004	32 $\rightarrow$ 20
(14, 5)	45	25	0.9095	45 $\rightarrow$ 25
(16, 6)	60	27	0.9155	60 $\rightarrow$ 27

Para (10, 4):

$$\boxed{30_{\text{full}} \rightarrow 24_{\text{red}} \rightarrow 16_{\text{J90}} .}$$

6.10 Passo 10 — nova coordenada $\mathbf{q}$

A restrição geométrica é

$$\boxed{\Delta\phi = Q_{90}\mathbf{q},}$$

com

$$\mathbf{q} \in \mathbb{R}^{r_{90}} .$$

Portanto, em vez de otimizar $k(n - k)$ números, otimizamos apenas r_{90} .

6.11 Passo 11 — construir J90-DIRECT

O circuito pode ser construído diretamente em termos de $\mathbf{q}$.

Para o motif associado ao parâmetro j :

$$\boxed{\theta_j(\mathbf{q}) = \theta_j^D + \frac{T_j}{2\pi} \sum_{a=1}^{r_{90}} (Q_{90})_{ja} q_a .}$$

Assim,

$$\boxed{U_{J90}(\mathbf{q}) = U_{\text{red}}(\phi_D + Q_{90}\mathbf{q}) .}$$

6.12 Passo 12 — nunca confundir ordem geométrica com ordem do circuito

Esta etapa é crítica para reproduzir corretamente o resultado.

Devemos registrar duas ordens diferentes:

- `theta_index`: posição do parâmetro na base onde Q_{90} foi calculado;
- `circuit_order`: ordem temporal real dos motifs no circuito.

O circuito deve ser reconstruído em

`circuit_order,`

mas cada motif consulta a linha de Q_{90} correspondente a seu

`theta_index.`

Pseudocódigo:

```
temporal = structure.sort_values("circuit_order")

for row in temporal.itertuples():
    j = row.theta_index

    expr = theta_D[j]

    for a in range(r90):
        expr += (
            period[j] / (2*np.pi)
            * Q90[j, a]
            * q[a]
        )

    append_same_physical_motif(expr)
```

6.13 Passo 13 — hard gate de equivalência

Antes de otimizar, teste obrigatoriamente:

$$U_{J90}^{\text{direct}}(\mathbf{q}) \stackrel{?}{=} U_{\text{red}}(Q_{90}\mathbf{q}).$$

Escolha:

- $\mathbf{q} = 0$;
- deslocamentos locais;
- vetores globais aleatórios.

Calcule os dois estados e exija:

$$1 - \mathcal{F} \leq 10^{-12},$$

$$D_{\text{TV}} \leq 10^{-12},$$

e erro máximo de amplitude, depois de alinhar a fase global,

$$\epsilon_{\text{amp}} \leq 10^{-10}.$$

Na execução corrigida, os resíduos permaneceram na escala numérica, aproximadamente 10^{-15} – 10^{-14} .

Isso certifica que J90-DIRECT não criou um circuito físico diferente.

6.14 Passo 14 — entender o que foi e não foi reduzido

Em J90,

o número de parâmetros livres diminui,

mas

a sequência de motifs físicos é a mesma do REDUCED.

Por exemplo, para (16, 6):

$$60 \rightarrow 27$$

graus de liberdade,

mas, após transpilation nível 3,

$$740 \text{ CX} \rightarrow 740 \text{ CX}.$$

Portanto J90 é compressão do espaço de busca, não gate pruning.

7 Benchmark necessário depois de cada redução

Reduzir dimensão não é suficiente.

É necessário verificar se a otimização continua funcionando.

O protocolo adotado é:

$$120 \text{ circuit-batches/run}$$

com

$$10\,000 \text{ shots/batch}.$$

Logo,

$$1.2 \times 10^6 \text{ shots/run}.$$

São usados:

$$112$$

batches para otimização e

$$8$$

batches independentes para confirmação final.

7.1 Otimizadores usados

A comparação entre arquiteturas foi repetida com:

- COBYLA;
- CMA-ES;
- SPSA;
- Powell;
- Nelder–Mead.

O ponto importante é sempre comparar:

mesmo otimizador + mesmo problema + mesmo start + mesmo orçamento.

7.2 Efeito de FULL → REDUCED

Para cada otimizador calculamos

$$\Delta C_{\text{struct}} = C_{\text{REDUCED}} - C_{\text{FULL}}.$$

Valor negativo favorece REDUCED.

O resultado principal é que o pruning estrutural reduziu recursos sem produzir penalidade sistemática.

Nelder–Mead apresentou ganho robusto.

COBYLA apresentou tendência favorável.

CMA-ES e SPSA ficaram essencialmente neutros nesse contraste.

Portanto:

REDUCED é o circuito-base recomendado para a frente atual.

7.3 Efeito de REDUCED → J90

Definimos

$$\Delta C_{J90} = C_{J90} - C_{\text{REDUCED}}.$$

No start Dicke:

Otimizador	win J90	mediana ΔC	leitura
COBYLA	73.3%	−0.00567	tendência favorável
CMA-ES	66.7%	−0.02387	tendência favorável
SPSA	10.0%	+0.01133	J90 piora
Powell	66.7%	−0.04285	tendência favorável
Nelder–Mead	100%	−0.07676	ganho robusto

Isso mostra que a redução geométrica deve ser escolhida em conjunto com o otimizador.

8 A triangulação final da redução

A apresentação pode ser organizada por três objetos:

$$N_{\text{nominal}} = k(n - k) + \binom{k}{2},$$

$$N_{\text{structural}} = k(n - k),$$

e

$$N_{\text{geometric}} = r_{90}.$$

A primeira passagem,

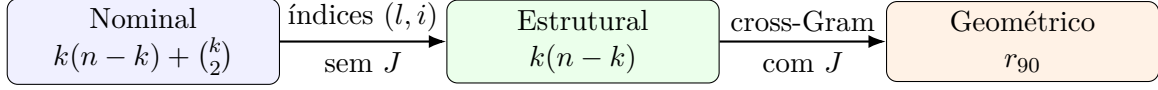
$$N_{\text{nominal}} \rightarrow N_{\text{structural}},$$

usa somente a arquitetura.

A segunda,

$$N_{\text{structural}} \rightarrow N_{\text{geometric}},$$

usa informação local do Jacobiano.
Podemos representar a lógica como:



Para (10, 4):

$$30 \rightarrow 24 \rightarrow 16.$$

Para (16, 6):

$$75 \rightarrow 60 \rightarrow 27.$$

9 Checklist

Antes de declarar uma execução válida, marque todos os itens.

Parte A — REDUCED sem Jacobiano

- ☐ A1. Fixei (n, k) .
- ☐ A2. Construí $\mathcal{O} = \{n - k, \dots, n - 1\}$.
- ☐ A3. Construí o FULL sem pruning.
- ☐ A4. Registre (l, i) e `circuit_order`.
- ☐ A5. Marquei candidatos mortos por $l, i \in \mathcal{O}$.
- ☐ A6. Obtive exatamente $\binom{k}{2}$ candidatos.
- ☐ A7. Obtive exatamente $k(n - k)$ motifs restantes.
- ☐ A8. Comparei com $\deg J(n, k) = k(n - k)$.
- ☐ A9. Randomizei somente candidatos mortos.
- ☐ A10. A distribuição permaneceu invariante dentro da tolerância.
- ☐ A11. Perturbei uma direção ativa como controle positivo.
- ☐ A12. Construí o REDUCED realmente removendo os motifs.
- ☐ A13. Transpilei FULL e REDUCED na mesma base.
- ☐ A14. Registre `CX`, `depth` e número de parâmetros.

Parte B — J90

- ☐ B1. Parti somente do REDUCED.
- ☐ B2. Normalizei as coordenadas angulares.
- ☐ B3. Fixei o Dicke como ponto de referência.
- ☐ B4. Estimei cada coluna de J por diferenças centradas.
- ☐ B5. Usei shots, não probabilidades exatas, para construir o J operacional.
- ☐ B6. Gerei duas réplicas independentes.
- ☐ B7. Construí G_{cross} .
- ☐ B8. Repeti em várias seeds.
- ☐ B9. Calculei $\overline{G}_{\text{cross}}$.
- ☐ B10. Projetei no cone PSD.
- ☐ B11. Calculei r_{90} .
- ☐ B12. Construí Q_{90} .
- ☐ B13. Criei apenas r_{90} parâmetros q_a .
- ☐ B14. Preservei `circuit_order`.
- ☐ B15. Usei `theta_index` apenas para selecionar a linha correta de Q_{90} .
- ☐ B16. Certifiquei $U_{J90}^{\text{direct}}(q) = U_{\text{red}}(Q_{90}q)$.
- ☐ B17. Só depois executei os otimizadores.

10 Algoritmo resumido

```
# =====  
# A. REDUCAO ESTRUTURAL -- NAO USA JACOBIANO  
# =====  
occupied_start = n - k  
  
def dead_pair(l, i):  
    return (  
        l >= occupied_start  
        and i >= occupied_start  
    )  
  
full = build_full(n, k)  
  
dead = [  
    motif  
    for motif in full.motifs  
    if dead_pair(motif.l, motif.i)  
]  
  
assert len(dead) == comb(k, 2)  
  
reduced = build_circuit(  
    skip=lambda motif: dead_pair(
```

```

        motif.l, motif.i
    )
)

assert reduced.num_parameters == k*(n-k)

# validar por variacoes aleatorias somente no setor dead
validate_dead_invariance(full)

# =====
# B. COMPRESSAO J90 -- USA JACOBIANO SHOT-BASED
# =====
J_replicates = []

for seed in architecture_seeds:

    JA = estimate_jacobian_shots(
        reduced,
        seed=(seed, "A"),
        step=0.08,
        shots=10000,
    )

    JB = estimate_jacobian_shots(
        reduced,
        seed=(seed, "B"),
        step=0.08,
        shots=10000,
    )

    Gcross = 0.5 * (
        JA.T @ JB
        + JB.T @ JA
    )

    save(Gcross)

Gmean = mean(all_Gcross)
Gplus = psd_project(Gmean)

eigvals, eigvecs = eigh(Gplus)
sort_descending()

r90 = first_r_with_cumulative_energy(0.90)
Q90 = eigvecs[:, :r90]

j90 = build_direct_j90(
    reduced_structure,
    Q90,
    preserve_circuit_order=True,
)

certify_equivalence(
    j90_direct=j90,
    reduced_projected=reduced,
    Q90=Q90,
)

```

11 resultado

A apresentação deve separar claramente:

1. Predição estrutural sem derivadas:

$$N_{\text{dead}} = \binom{k}{2}, \quad N_{\text{red}} = k(n - k).$$

2. Validação física do pruning: variar o setor candidato morto não altera o estado/distribuição dentro da precisão do teste.

3. Economia de circuito: FULL $\rightarrow$ REDUCED reduz parâmetros, CX e depth.

4. Compressão geométrica: REDUCED $\rightarrow$ J90 reduz

$$k(n - k) \rightarrow r_{90}$$

graus de liberdade.

5. Certificação da implementação J90:

$$U_{J90}^{\text{direct}}(q) = U_{\text{red}}(Q_{90}q).$$

6. Benchmark por otimizador: J90 não é tratado como universal; o efeito é medido separadamente para cada algoritmo.

12 Conclusão

A redução do circuito ocorre em duas camadas.

A primeira camada é estrutural e pode ser identificada antes de qualquer Jacobiano:

$$k(n - k) + \binom{k}{2} \longrightarrow k(n - k).$$

Sua hipótese pode ser formulada diretamente da topologia do ansatz e é reforçada pela triangulação combinatória

$$N_{\text{red}} = \deg J(n, k) = k(n - k).$$

Nos regimes testados,

$$(8, 3), \quad (10, 4), \quad (12, 4), \quad (14, 5), \quad (16, 6),$$

essa regra produziu exatamente a contagem usada no circuito REDUCED.

A segunda camada é geométrica:

$$k(n - k) \longrightarrow r_{90}.$$

Ela exige medir o Jacobiano, construir um cross-Gram multi-seed e certificar que o circuito J90-DIRECT é fisicamente a mesma família restrita

$$\Delta\phi = Q_{90}q.$$

A leitura final é:

$$\text{FULL} \rightarrow \text{REDUCED} \rightarrow \text{J90-DIRECT}$$

com

pruning estrutural $\rightarrow$ compressão geométrica.

Para a continuação do projeto, o REDUCED deve ser tratado como circuito-base.
O J90 deve ser ativado somente quando o benchmark do otimizador justificar a compressão.